

Tema 6 - Algoritmos Recursivos

En el **Tema 6** del libro *Fundamentos de algoritmia* de Brassard y Bratley se desarrolla uno de los conceptos más importantes dentro del estudio de algoritmos: la **recursión**. Este capítulo explica cómo algunos problemas se pueden resolver definiéndolos en términos de sí mismos, es decir, un **algoritmo recursivo** es aquel en el cual una función se invoca a sí misma para alcanzar la solución del problema.

Conceptos principales

La recursión permite **dividir un problema en subproblemas más pequeños** hasta llegar a un caso base o condición de parada, que es la situación más simple que puede resolverse sin necesidad de llamadas adicionales. Una recursión correctamente definida siempre debe tener este **caso base** para evitar llamadas infinitas.

Se identifican dos tipos de recursión:

- **Recursión directa:** una función se llama a sí misma directamente.
- **Recursión indirecta:** una función A llama a una función B, que eventualmente vuelve a llamar a A.

Análisis de la eficiencia

El tema también se centra en cómo calcular la eficiencia de algoritmos recursivos mediante **ecuaciones de recurrencia**, donde se expresa el costo del algoritmo en función del tamaño del problema. Se estudian métodos para resolver estas ecuaciones, como el **método de sustitución**, y se muestra cómo la recursión puede ser menos eficiente que los algoritmos iterativos, principalmente debido al uso de memoria adicional por las llamadas recursivas.

Eliminación de la recursión

El capítulo explica que, aunque la recursión es elegante y simplifica el diseño de algunos algoritmos, muchas veces es posible transformar una solución recursiva en una versión **iterativa**, lo cual permite mejorar el rendimiento del programa, especialmente en problemas que requieren gran cantidad de llamadas recursivas.

Ejemplos Prácticos

1. Secuencia de Fibonacci

El clásico ejemplo de recursión es el cálculo del **n-ésimo término de la secuencia de Fibonacci**, donde cada término es la suma de los dos anteriores:

```
Fibonacci(n):  
  Si n == 0 o n == 1 entonces  
    retornar n  
  Sino  
    retornar Fibonacci(n-1) + Fibonacci(n-2)
```

Este ejemplo es sencillo pero ineficiente, ya que realiza muchas llamadas redundantes. Es un buen caso para aprender optimización de recursión mediante **memoización** o su conversión a un algoritmo iterativo.

2. Búsqueda Binaria

La **búsqueda binaria** es otro ejemplo clásico, en el cual un arreglo ordenado se divide a la mitad en cada paso para encontrar un valor específico:

```
BusquedaBinaria(arr, inicio, fin, x):  
  Si inicio > fin entonces  
    retornar -1  
  medio = (inicio + fin) / 2  
  Si arr[medio] == x entonces  
    retornar medio  
  Si arr[medio] > x entonces  
    retornar BusquedaBinaria(arr, inicio, medio-1, x)  
  Sino  
    retornar BusquedaBinaria(arr, medio+1, fin, x)
```

Este algoritmo es muy eficiente, con una complejidad de **$O(\log n)$** , y demuestra cómo la recursión es útil para problemas de división y conquista.

3. Torres de Hanoi

El **problema de las Torres de Hanoi** es un clásico problema recursivo en el que se deben mover discos entre tres torres respetando ciertas reglas:

```
TorresHanoi(n, origen, destino, auxiliar):  
  Si n == 1 entonces  
    mover disco de origen a destino  
  Sino  
    TorresHanoi(n-1, origen, auxiliar, destino)  
    mover disco de origen a destino  
    TorresHanoi(n-1, auxiliar, destino, origen)
```

La solución recursiva tiene una complejidad de **$O(2^n)$** y ejemplifica cómo la recursión permite resolver problemas complejos con pocas líneas de código.